

单例模式 + Spring 单例管理 完整复习笔记

整理自 MRDP 项目复习对话（章节 5 缓存 DCL 引出的横向知识点） 适用于：Java 后端面试、Spring 原理深挖

目录

- 一、概念澄清
- 二、什么是单例模式
- 三、单例模式 7 个版本演进
- 四、版本对比速查表
- 五、Spring 是怎么管理单例 Bean 的
- 六、循环依赖与三级缓存
- 七、项目里的实际应用
- 八、面试金句汇总
- 九、一道趣味题：反射破坏单例
- 十、知识闭环图

一、概念澄清

 **Singleton** 不是 Java 关键字！

```
public class Singleton {    // ← 这是个普通类名
    ...
}
```

Java 真正的关键字约 50 个，包括：`class`、`public`、`private`、`static`、`final`、`synchronized`、`volatile`、`new`、`return`、`if`、`for`、`while` 等。

Singleton、**String**、**List** 这些都是普通类名——只是因为太常用看着眼熟。理论上你可以把单例类叫 **XiaoMing**，照样能写。

🌟 **关键字鉴别方法**：在 IDE 里关键字会有特殊颜色（通常是蓝色或紫色加粗），类名/方法名是另一种颜色。

二、什么是单例模式

一句话定义

保证一个类在整个 JVM 里只有一个实例，并提供全局访问点。

现实生活中的"单例"类比

类比	说明
 公司只有一个 CEO	CEO 类是单例
 地球只有一个地球	Earth 类是单例
 操作系统只有一个文件系统	FileSystem 类是单例

编程中什么时候要单例？

场景	为什么单例
数据库连接池	创建连接很贵，全应用共享一份
配置管理器	配置文件全局一份
日志记录器（Logger）	全应用一份，避免日志混乱
Spring Bean（默认）	<code>@Service</code> 、 <code>@Component</code> 默认单例
项目里的 <code>CACHE_REBUILD_EXECUTOR</code>	整个 <code>CacheClient</code> 共享一个线程池

你已经写过的单例

```
// CacheClient.java
public static final ExecutorService CACHE_REBUILD_EXECUTOR =
    Executors.newFixedThreadPool(10);
```

整个 `CacheClient` 类的所有实例**共享同一个线程池**——这就是 `static` 字段的本质：“**全 JVM 只有一份**”。

三、单例模式 7 个版本演进

V1：饿汉式（Eager Initialization）

```
public class Singleton {
    // 类加载时就创建实例
    private static final Singleton INSTANCE = new Singleton();

    // 私有构造函数：阻止外界 new
    private Singleton() {}

    public static Singleton getInstance() {
        return INSTANCE;
    }
}

// 用法
Singleton s = Singleton.getInstance();
```

关键点：

- `private` 构造函数 → 外面 `new Singleton()` 编译报错
- `static final INSTANCE` → 类加载时立即创建，不可改

优点	缺点
✅ 线程安全（JVM 保证类加载是原子的）	❌ 类加载就创建，可能浪费
✅ 简单	❌ 不支持懒加载

V2：懒汉式（Lazy Initialization）—— 线程不安全

```
public class Singleton {
    private static Singleton instance;    // 不立即创建
    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) {           // ← 用到时才创建
            instance = new Singleton();
        }
        return instance;
    }
}
```

优点：✅ 懒加载（用时才创建） 致命缺点：❌ 多线程下会创建多个实例

多线程 Bug 演示

时刻	Thread A	Thread B
T1	if (instance == null) → true	
T2		if (instance == null) → true
T3	instance = new Singleton() （创建实例 1）	
T4		instance = new Singleton() （创建实例 2）
T5	return 实例1	
T6		return 实例2

结果：JVM 里有 2 个实例 → 不再是单例！

V3：synchronized 方法 —— 简单粗暴

```
public static synchronized Singleton getInstance() {
    if (instance == null) {
        instance = new Singleton();
    }
}
```

```

        return instance;
    }

```

加 **synchronized** 关键字让整个方法变成互斥的——同一时刻只有一个线程能进。

优点：✅ 线程安全 **缺点：**❌ 每次调用都要加锁——但 99.99% 的调用 instance 都已经创建好了，**不需要互斥！**

性能很差，每次 **getInstance()** 都要争锁。

V4：DCL（双重检查锁）—— 关键登场 🎯

```

public class Singleton {
    private static Singleton instance;    // ⚠️ 注意：还没加 volatile
    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) {          // ★ 第一次检查（无锁）
            synchronized (Singleton.class) { // 类锁
                if (instance == null) {    // ★ 第二次检查（有锁）
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}

```

思想：只在 instance 还没创建时才争锁，已创建后无锁直接返回。

99.99% 的调用都走“第一次检查 → 直接 return”，几乎没有锁开销。

只有在**实例尚未创建的极少数瞬间**才进入 synchronized，**还要再检查一次**（防止刚才两个线程都通过第一检查，都进入临界区，避免重复创建）。

和缓存 DCL 同源

缓存 DCL (CacheClient):
 外层无锁判过期 → 抢锁后 → 内层再判一次（防别人已经重建）

单例 DCL:
 外层无锁判 null → 抢锁后 → 内层再判一次（防别人已经创建）

两个场景的核心问题都是“判断和动作之间状态可能变化”。

🔥 V4 还有个隐藏的 Bug —— 指令重排序

`instance = new Singleton()` 这一行看似一步操作，JVM 实际分 3 步：

- 步骤 1：在堆上分配内存
- 步骤 2：调用构造函数初始化对象（设置字段值等）
- 步骤 3：把对象的引用地址赋给 `instance` 变量

JVM 和 CPU 出于性能考虑，可能把这 3 步重排序为 $1 \rightarrow 3 \rightarrow 2$ 。

重排序后：

- 步骤 1：分配内存（地址 `0x1234`）
- 步骤 3：`instance = 0x1234`（此时对象还没初始化！）
- 步骤 2：调用构造函数初始化（字段还都是默认值）

如果在步骤 3 和步骤 2 之间另一个线程进来：

```
Thread A 执行到步骤 3 完成 (instance != null 了，但对象还没初始化)
      ↓
Thread B: if (instance == null) → false
          return instance      ← 拿到一个"半成品"对象！
          访问 instance.someField → 可能拿到默认值或抛异常
      ↓
Thread A 才执行步骤 2（初始化对象）
```

Thread B 读到了未完全构造的对象——经典并发 Bug。

V5：DCL + `volatile` —— 真正的最终版

```
public class Singleton {
    private static volatile Singleton instance;    // ★ 加 volatile
    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) {
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
```

`volatile` 关键字的两大作用：

作用	说明
禁止指令重排序	<code>instance = new Singleton()</code> 三步必须按 1→2→3 顺序执行
保证可见性	一个线程修改后，其他线程立即可见（不会缓存到 CPU 寄存器）

✦ "DCL 必须配 **volatile**" —— Java 面试经典考点，答出"volatile 防止指令重排序"瞬间加分。

V6：静态内部类（推荐方案 1）

```
public class Singleton {
    private Singleton() {}

    // 内部静态类
    private static class Holder {
        static final Singleton INSTANCE = new Singleton();
    }

    public static Singleton getInstance() {
        return Holder.INSTANCE;
    }
}
```

精妙之处：

- `Holder` 类只有第一次被引用时才会加载——这是 JVM 的特性
- `Holder.INSTANCE = new Singleton()` 在类加载时执行，JVM 保证这一步线程安全
- 不需要 `volatile`、不需要 `synchronized`、不需要 DCL
- 既懒加载又线程安全

V7：枚举（推荐方案 2，《Effective Java》强推）

```
public enum Singleton {
    INSTANCE;

    public void doSomething() {
        // 业务方法
    }
}

// 用法
Singleton.INSTANCE.doSomething();
```

为什么是终极方案：

优势	说明
线程安全	JVM 保证 enum 实例只创建一次
防反射攻击	反射 <code>Constructor.newInstance()</code> 在 enum 上会抛 <code>IllegalArgumentException</code>
防反序列化破坏	enum 的反序列化会从 <code>valueOf</code> 返回已有实例，不会创建新的
代码极简	一行搞定

缺点：

- 不能延迟加载（enum 是饿汉式）
- 不能继承

🌸 《Effective Java》Joshua Bloch（Java 集合框架作者）："单例首选 enum"。

四、版本对比速查表

版本	懒加载	线程安全	性能	防反射	推荐度
V1 饿汉式	✗	✓	✓	✗	★★★
V2 懒汉式	✓	✗	✓	✗	✗
V3 synchronized 方法	✓	✓	✗	✗	✗
V4 DCL（无 volatile）	✓	✗	✓	✗	✗
V5 DCL + volatile	✓	✓	✓	✗	★★★★
V6 静态内部类	✓	✓	✓	✗	★★★★★
V7 枚举	✗	✓	✓	✓	★★★★★

五、Spring 是怎么管理单例 Bean 的

Bean 的"作用域"（Scope）

Spring 支持 5 种作用域：

作用域	关键字	含义
singleton ★	<code>@Scope("singleton")</code>	默认，全容器一个实例
prototype	<code>@Scope("prototype")</code>	每次 <code>getBean</code> 创建新实例
request	<code>@Scope("request")</code>	每个 HTTP 请求一个（Web 专用）
session	<code>@Scope("session")</code>	每个 HTTP Session 一个（Web 专用）
application	<code>@Scope("application")</code>	每个 <code>ServletContext</code> 一个

99% 的 Spring Bean 都是 singleton——所有 `@Service`、`@Component`、`@Controller`、`@Repository` 默认全是单例。

✦ **何时用 prototype?** 当 Bean 持有可变状态（如表单填写中间态对象、计数器），不能共享。

单例 Bean 存在哪? —— "单例池"

Spring 容器 (`ApplicationContext`) 内部维护一个 `ConcurrentHashMap`:

```
// 源码: org.springframework.beans.factory.support.DefaultSingletonBeanRegistry
private final Map<String, Object> singletonObjects =
    new ConcurrentHashMap<>(256);
```

这就是"单例池" —— Spring 容器启动时把所有 singleton Bean 创建好放这里，调用 `getBean("xxx")` 直接从这个 Map 拿。

singletonObjects (单例池)

"userServiceImpl" → UserServiceImpl@0x1234

"voucherOrderService" → VoucherOrderServiceImpl@0x5678

"cacheClient" → CacheClient@0x9abc

"shopMapper" → ShopMapper代理对象@0xdef0

...

整个 JVM 里 Spring 管的所有单例 Bean 都在这一个 Map 里

和静态内部类对比

维度	静态内部类	Spring 单例池
实现	JVM 类加载机制	<code>ConcurrentHashMap</code>
线程安全	✅ JVM 保证	✅ <code>ConcurrentHashMap</code> 保证
懒加载	✅	默认饿汉（启动就创建），可配 <code>lazy-init</code>
灵活性	❌ 一个类一份代码	✅ 可配置依赖、AOP 代理、生命周期

Spring 选了"重武器": 本质还是个线程安全的 Map，但它需要管理依赖注入、AOP、生命周期，比单纯的"单例"复杂得多。

Bean 的生命周期

① 实例化 (Instantiation)

调用构造函数: `new UserServiceImpl()`

此时 Bean 是"半成品" — 字段还都是 `null`

↓

② 属性填充 (Population)

处理 @Autowired / @Resource
向 Bean 里塞依赖 (其他 Bean)

↓

③ 初始化 (Initialization)

调用 @PostConstruct 方法
调用 InitializingBean.afterPropertiesSet()
执行 BeanPostProcessor 的 after 方法 (AOP 在此织入)

↓

④ 放进单例池

singletonObjects.put(beanName, bean)

↓

Bean 可被使用

↓

... 应用运行中 ...

↓

⑤ 销毁 (Destruction)

容器关闭时调用 @PreDestroy 方法
调用 DisposableBean.destroy()

✦ **关键点:** Bean 是"半成品 → 成品"逐步组装的, 不是一蹴而就。

六、循环依赖与三级缓存

循环依赖场景

```
@Service
public class A {
    @Autowired
    private B b;    // A 需要 B
}

@Service
public class B {
    @Autowired
    private A a;    // B 需要 A
}
```

Spring 启动时会发生什么?



没有特殊处理就是无限递归 → StackOverflowError。

三级缓存解决方案

Spring 不是只有 1 个单例池，而是 **3 层缓存**：

```
// org.springframework.beans.factory.support.DefaultSingletonBeanRegistry

// 一级缓存：完整的 Bean
private final Map<String, Object> singletonObjects = new ConcurrentHashMap<>(256);

// 二级缓存：早期 Bean（实例化了但还没填属性）
private final Map<String, Object> earlySingletonObjects = new HashMap<>(16);

// 三级缓存：Bean 工厂（用来生成早期引用，可能是代理对象）
private final Map<String, ObjectFactory<?>> singletonFactories = new HashMap<>(16);
```

三级缓存解决循环依赖的流程

时刻 存	动作 三级缓存	一级缓存	二级缓
T1	开始创建 A		

```

T2  A 实例化 (new A()) — 半成品
A的工厂
T3  A 开始属性填充, 发现需要 B
T4  去找 B → 三级都没有 → 创建 B
T5  B 实例化 (new B()) — 半成品
B的工厂
T6  B 开始属性填充, 发现需要 A
T7  去找 A → 一级没有 → 二级没有 → 三级有!           A实例(半
成品)
      从三级取出工厂调用, 生成"早期 A 引用", 升到二级
T8  B.a = 这个早期 A 引用 ✓
T9  B 完成所有阶段 (初始化、AOP)           B(完整)
T10 B 进一级缓存                               删 B 的
      二级
T11 回到 A, A.b = B (从一级取)
T12 A 完成所有阶段           A(完整)           删 A 的
      二级           删 A 的三级
T13 A 进一级缓存

```

核心思想：让"半成品"对象先暴露出去，等所有依赖填完后再变成完整品。

为什么需要三级，二级不够吗？

这是面试官最爱深挖的问题。

假设只有二级缓存（直接存早期对象）

二级缓存存的是 **A 实例(半成品)** —— 一个**真实对象**的引用。

问题：如果 A 需要被 AOP 代理（比如标了 `@Transactional`），最终用户拿到的应该是**代理对象**而不是原始对象。

```

@Service
public class A {
    @Transactional
    public void doSomething() { ... }
}

```

二级缓存方案的 Bug：

```

T6: B 拿走的是"原始 A 引用" (A 实例)
T9-T12: A 完成 AOP 织入, 生成"代理 A"
T13: 一级缓存里是"代理 A", 但 B 持有的是"原始 A"!

```

结果：B 调用 `a.doSomething()` 不走代理 → 事务不生效！

三级缓存怎么解决？

三级缓存存的是 **ObjectFactory<?>**（一个工厂方法），不是直接的对象。

```
singletonFactories.put("A", () -> {
    // 这个 lambda 在被调用时执行：
    if (A 需要 AOP 代理) {
        return 代理 A;
    } else {
        return 原始 A;
    }
});
```

当 B 来取 A 时：

- T7：调用 A 的 `ObjectFactory.getObject()`
- 工厂判断 A 是否需要代理
 - 需要：返回代理 A（提前生成代理！）
 - 不需要：返回原始 A
 - 把生成的对象升到二级缓存（避免下次又生成新的）

✦ **三级缓存的精髓：用"延迟生成"应对"未知需求"**——在 Bean 完全成型前，无法预知它是否需要代理。三级缓存留了一个"动态决定生成什么"的钩子。

三级缓存职责对照表

缓存	名称	存什么	何时存入	何时取出
一级	singletonObjects	完整成品 Bean	整个生命周期跑完后	正常 getBean
二级	earlySingletonObjects	半成品 Bean / 早期代理	三级被调用一次后升级	循环依赖里第二次访问
三级	singletonFactories	Bean 工厂 (lambda)	Bean 实例化后立即放入	循环依赖里第一次访问

三级缓存的局限性

限制 1：只解决 **setter / 字段注入**的循环依赖

```
// ✅ 能解决（字段注入）
public class A {
    @Autowired private B b;
}
```

```
// ✅ 能解决 (setter 注入)
public class A {
    private B b;
    @Autowired public void setB(B b) { this.b = b; }
}

// ❌ 解决不了 (构造器注入)
public class A {
    private final B b;
    @Autowired public A(B b) { this.b = b; }
}
```

为什么构造器循环依赖解决不了？因为 `new A()` 时就需要 B 实例，连“半成品 A”都创建不出来——三级缓存没机会暴露半成品。

✦ **Spring 推荐构造器注入**（更安全），但会暴露循环依赖——通常这是好事，循环依赖往往是设计有问题！

限制 2：不解决 **prototype** 作用域

prototype Bean 每次都创建新实例，三级缓存不缓存它，循环依赖直接报错。

七、项目里的实际应用

场景 1：UserServiceImpl

```
@Service
public class UserServiceImpl extends ServiceImpl<UserMapper, User> implements
    IUserService {
    @Resource
    StringRedisTemplate stringRedisTemplate;
}
```

Spring 启动时：

1. 创建 UserServiceImpl 实例（半成品，stringRedisTemplate 是 null）
2. 三级缓存放入 UserServiceImpl 的工厂
3. 属性填充：找 stringRedisTemplate → 已在容器中（Spring Boot 自动配置）→ 注入
4. 初始化（无 @PostConstruct）
5. 一级缓存里有 UserServiceImpl 完整版

场景 2：自注入（之前删掉的那个）

```
@Autowired
private IUserService iUserService; // ← 自己注入自己
```

为什么这能成功？ 因为 Spring 的三级缓存机制，"半成品 UserServiceImpl"能被自己的属性引用。**Spring 在这里做了特殊处理**——这就是单例 Bean 自注入的内在原理。

场景 3：AopContext.currentProxy()（章节 6 详讲）

```
IVoucherOrderService proxy = (IVoucherOrderService) AopContext.currentProxy();
proxy.createVoucherOrder(voucherId);    // ← 通过代理调用，事务才生效
```

为什么不能直接 this.createVoucherOrder()？ → 因为 this 是原始对象，绕过了 AOP 代理——事务、缓存等切面都不会触发。

这就是 Spring 单例管理 + AOP 代理的 trade-off：

- 单例池里存的是"代理对象"
- 但 Bean 内部的 this 是"原始对象"
- 想触发代理，必须通过容器拿代理对象或用 AopContext.currentProxy()

场景 4：CacheClient.CACHE_REBUILD_EXECUTOR

```
public static final ExecutorService CACHE_REBUILD_EXECUTOR =
    Executors.newFixedThreadPool(10);
```

虽然 CacheClient 类是 Spring 单例下只有 1 个实例，但 static 字段意味着：**即使有多个 CacheClient 实例，这个线程池也只有一份。**

八、面试金句汇总

基础题：单例模式怎么实现？

"单例的核心是**私有构造函数 + 静态访问方法**。最简单是饿汉式，类加载时创建，线程安全但不懒加载。

想懒加载就得用懒汉式——但简单的 if-null 判断在多线程下会创建多个实例，所以经典做法是 **DCL（双重检查锁）**：外层无锁判 null，进入 synchronized 后再判一次，这样 99.99% 的调用走无锁快速路径。

但 DCL 必须配 **volatile**——因为 new Singleton() 实际是分配内存→初始化→赋引用三步，JVM 可能重排序，导致另一个线程读到未完成构造的半成品对象。volatile 既禁止重排序也保证可见性。

不过现代写法我更推荐**静态内部类**——利用 JVM 类加载机制天然线程安全 + 懒加载，代码更干净；或者**枚举**，《Effective Java》推荐，还能防反射攻击和反序列化破坏。"

基础题：Spring 单例怎么管理？

"Spring 容器内部用一个叫 singletonObjects 的 ConcurrentHashMap 缓存所有单例 Bean。
@Service、@Component 默认都是 singleton 作用域，启动时容器就把它们创建好放进去，@Autowired 注入时直接从这个 Map 拿。"

进阶题：循环依赖怎么解决？

"Spring 用**三级缓存**解决 setter/字段注入的循环依赖：

- 一级 `singletonObjects` 存完整 Bean
- 二级 `earlySingletonObjects` 存早期对象
- 三级 `singletonFactories` 存 Bean 工厂

Bean 实例化完先把工厂放三级。被另一个 Bean 引用时调工厂生成早期引用（可能是代理）升到二级，让对方先持有这个引用继续完成构造，自己的属性填完后再升到一级。

构造器循环依赖解决不了，因为连半成品都建不出来——这恰恰说明你的设计有问题。"

深挖题：为什么三级而不是二级？

"二级缓存只能存对象引用。如果 Bean 需要 AOP 代理，二级方案下别人拿到的是**原始对象**，错过代理织入。三级缓存存的是**工厂方法**，被调用时可以**动态判断**要返回原始对象还是代理对象，**保证最终引用的一致性**。"

九、一道趣味题：反射破坏单例

```
public class Singleton {  
    private static Singleton instance;  
    private Singleton() {}  
    public static Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

面试官问：这段代码用反射能拿到第二个实例吗？

提示：`Constructor.setAccessible(true)` 能突破 private 限制

答案：可以！

```
Constructor<Singleton> c = Singleton.class.getDeclaredConstructor();  
c.setAccessible(true); // 突破 private  
Singleton s1 = Singleton.getInstance();  
Singleton s2 = c.newInstance(); // ★ 反射创建第二个实例  
System.out.println(s1 == s2); // false ✗
```

怎么防？ 在私有构造函数里再加一层校验：

```
private Singleton() {
    if (instance != null) {
        throw new RuntimeException("不能反射创建第二个实例");
    }
}
```

但这还能被绕（多次反射），所以 **enum 才是终极方案**——JVM 在反射 enum 时会主动抛异常，绕都绕不过。

十、知识闭环图

单例模式演进闭环

V1 饿汉式
↓ 想懒加载
V2 懒汉式（线程不安全）
↓ 加锁
V3 synchronized 方法（性能差）
↓ 缩小锁范围
V4 DCL（重排序问题）
↓ 加 volatile
V5 DCL + volatile ✓
↓ 嫌麻烦
V6 静态内部类 ✓✓
↓ 终极方案
V7 枚举 ✓✓✓

Bean 创建过程（带循环依赖）



知识点串联图

缓存 DCL (CacheClient)
↓ 同源思想
单例 DCL (V4 → V5)
↓ 引出 volatile
JMM 内存模型 + 指令重排序
↓ 引出更好的方案
静态内部类 (V6) / 枚举 (V7)

↓ 实战应用
 Spring 单例池 (ConcurrentHashMap)
 ↓ 引出循环依赖
 三级缓存 (singletonObjects / earlySingletonObjects / singletonFactories)
 ↓ 为什么需要三级
 AOP 代理一致性
 ↓ 实战应用
 AopContext.currentProxy() (章节 6 秒杀)

延伸阅读

- 《Effective Java》Joshua Bloch —— Item 3 强制单例性
- 《Java 并发编程实战》—— volatile 与 happens-before
- Spring 源码: [DefaultSingletonBeanRegistry](#)、[AbstractAutowireCapableBeanFactory](#)
- 经典文章: Doug Lea 《Double-Checked Locking is Broken》

自我检验清单

学完这一章，你应该能：

- ☐ 不看代码写出 V5 DCL + volatile 版本
- ☐ 解释为什么 V4 没 volatile 会有问题 (指令重排序)
- ☐ 说出 V6 静态内部类为什么线程安全 (JVM 类加载机制)
- ☐ 列出 V7 枚举的 4 个优势
- ☐ 说出 Spring 单例池的字段名 [singletonObjects](#) 和类型 [ConcurrentHashMap](#)
- ☐ 复述 Bean 生命周期 5 个阶段
- ☐ 解释三级缓存各自的存储内容和触发时机
- ☐ 回答“为什么不能用二级缓存代替三级” (AOP 代理一致性)
- ☐ 说出循环依赖解决不了的两种情况 (构造器注入 / prototype)
- ☐ 用反射演示如何破坏 V5 单例，并说出防御方案

写于 MRDP 项目复习途中。下一章: [VoucherOrderServiceImpl](#) 秒杀核心 + [AopContext.currentProxy\(\)](#) 实战。

附录 A: MyBatis-Plus 链式调用速通

整理自章节 6.1 乐观锁 SQL 翻译练习引出的横向知识点 适用于: 项目代码理解、面试 ORM 八股文

A.1 三个 ORM 框架对比

框架	哲学	写多少 SQL	复杂查询
MyBatis	半自动, SQL 你来写	大部分	适合 (自由)

📌 **一句话：**MyBatis-Plus = MyBatis + 简单 CRUD 的脚手架 + 链式 API。

为什么国内大量项目用它？

简单 CRUD 不用写一行 SQL——开发效率提升 50%+。

A.2 ServiceImpl 给你白送的魔法方法

单条操作

批量操作

18 / 42

链式入口（重头戏）

```
service.query()           // 启动一个查询链
service.update()          // 启动一个更新链
service.lambdaQuery()     // Lambda 版查询链
service.lambdaUpdate()    // Lambda 版更新链
```

A.3 链式调用的"4 步公式"

```
service.query()           // ① 起手：开始链
    .eq("name", "张三")   // ② 配置：拼条件
    .gt("age", 18)
    .one();               // ③ 终止：执行并取结果
```



起手：5 个入口

起手方法	返回什么	用途
query()	QueryChainWrapper	查询（字符串字段名）
lambdaQuery()	LambdaQueryChainWrapper	查询（Lambda 类型安全）
update()	UpdateChainWrapper	更新
lambdaUpdate()	LambdaUpdateChainWrapper	更新（Lambda）
removeBatchByIds()	(直接终止)	批量删除

A.4 配置阶段——常用条件方法速查表

比较类（最常用）

方法	含义	等价 SQL
eq(col, val)	equal	col = ?
ne(col, val)	not equal	col != ?
gt(col, val)	greater than	col > ?

方法	含义	等价 SQL
ge(col, val)	greater or equal	col >= ?
lt(col, val)	less than	col < ?
le(col, val)	less or equal	col <= ?
between(col, v1, v2)	范围	col BETWEEN ? AND ?
notBetween	范围外	col NOT BETWEEN ? AND ?

✦ 记忆法：

- e = equal、n = not、g = greater、l = less
- 加 e 表示 "or equal"（含等号）
- gt < ge < eq < le < lt 这套全套

集合类

方法	等价 SQL
in(col, c1, c2, c3)	col IN (?, ?, ?)
notIn(col, list)	col NOT IN (...)
inSql(col, "SELECT ...")	col IN (子查询)

模糊匹配（⚠ 易踩坑，下方 A.8 详解）

方法	等价 SQL
like(col, val)	col LIKE '%val%'
likeLeft(col, val)	col LIKE '%val'
likeRight(col, val)	col LIKE 'val%'
notLike(col, val)	col NOT LIKE '%val%'

空值

方法	等价 SQL
isNull(col)	col IS NULL
isNotNull(col)	col IS NOT NULL

排序 / 分组

方法	等价 SQL
orderByAsc(col)	ORDER BY col ASC
orderByDesc(col)	ORDER BY col DESC

方法	等价 SQL
<code>groupBy(col)</code>	<code>GROUP BY col</code>
<code>having(condition)</code>	<code>HAVING condition</code>

拼任意 SQL（终极大招）

方法	用途
<code>last("LIMIT 1")</code>	在 SQL 末尾追加任意片段
<code>apply("col = NOW()")</code>	拼自定义 WHERE 条件（注意 SQL 注入风险）

更新专用：set vs setSql

```
// set — 普通字段赋值
.set("name", "张三")           // SET name = '张三'
.set("age", 25)                 // SET age = 25

// setSql — 拼任意 SQL 片段（用于 stock = stock - 1 这种）
.setSql("stock = stock - 1")    // SET stock = stock - 1
.setSql("update_time = NOW()") // SET update_time = NOW()
```

✦ `setSql` 是 `setter` 不能表达的需求才用——比如自增、调用 SQL 函数、引用其他字段。

A.5 终止方法速查表

查询终止

方法	返回值	用途
<code>.one()</code>	T (实体)	期望 0 或 1 条；多条会抛异常
<code>.list()</code>	List	多条记录
<code>.count()</code>	long	数量
<code>.exists()</code>	boolean	是否存在
<code>.page(page)</code>	IPage	分页

更新/删除终止

方法	返回值	用途
<code>.update()</code>	boolean	执行 UPDATE
<code>.remove()</code>	boolean	执行 DELETE

A.6 实战：项目里的 5 个例子

例 1：乐观锁更新（VoucherOrderServiceImpl）

```
seckillVoucherService.update()           // ① 起手
    .setSql("stock = stock - 1")         // ② 配置 SET
    .eq("voucher_id", voucherId)         // ② 配置 WHERE 1
    .gt("stock", 0)                      // ② 配置 WHERE 2
    .update();                           // ③ 终止
```

翻译：

```
UPDATE tb_seckill_voucher
SET stock = stock - 1
WHERE voucher_id = ? AND stock > 0;
```

返回 boolean：true = 至少 1 行受影响（抢到了）；false = 0 行（库存没了）。

例 2：一人一单查询

```
// VoucherOrderServiceImpl.java
long count = query()
    .eq("user_id", userId)
    .eq("voucher_id", voucherId)
    .count();
```

翻译：

```
SELECT COUNT(*) FROM tb_voucher_order
WHERE user_id = ? AND voucher_id = ?;
```

✦ **注意：**因为本类继承了 `ServiceImpl<VoucherOrderMapper, VoucherOrder>`，所以 `query()` 不用写 `service` 前缀，直接调用——针对本类对应的 entity（`VoucherOrder`）。

例 3：登录功能查用户

```
// UserServiceImpl.java
User user = query().eq("phone", phone).one();
```

翻译：

```
SELECT * FROM tb_user WHERE phone = ?;
```

⚠️ **.one()** 的隐藏陷阱：如果数据库中匹配的记录**多于 1 条**，会抛 `TooManyResultsException`。只在你确信**"最多 1 条"**时用。

例 4：列表查询（假设场景）

```
// 查所有 status=1 且按销量排序的店铺
List<Shop> shops = shopService.query()
    .eq("status", 1)
    .orderByDesc("sold")
    .list();
```

翻译：

```
SELECT * FROM tb_shop WHERE status = 1 ORDER BY sold DESC;
```

例 5：分页

```
Page<Shop> page = new Page<>(1, 10);    // 第 1 页，每页 10 条

shopService.query()
    .eq("type_id", 1)
    .orderByDesc("score")
    .page(page);                        // ★ 终止用 .page(page)

List<Shop> records = page.getRecords(); // 当前页数据
long total = page.getTotal();          // 总记录数
```

✦ **分页要在配置类启用拦截器**——你项目的 `MybatisConfig.java` 里有：

```
interceptor.addInnerInterceptor(new PaginationInnerInterceptor(DbType.MYSQL));
```

A.7 Lambda 版本：类型安全升级

问题：字符串字段名容易写错

```
.eq("user_id", userId)           // 字段名手敲，可能拼错 ("users_id")  
                                // 编译器不报错 → 运行时挂
```

解决：Lambda 表达式

```
lambdaQuery()  
    .eq(VoucherOrder::getUserId, userId)    // 用方法引用!  
    .eq(VoucherOrder::getVoucherId, voucherId)  
    .count();
```

好处：

- ☒ 编译期检查（字段错了 IDE 标红）
- ☒ 重命名实体字段时自动跟着改
- ☒ IDE 自动补全

为什么黑马点评教学用字符串版？ 简洁，新手好懂。生产推荐 Lambda 版。

A.8 ⚠ 经典陷阱：likeLeft / likeRight / like

这是 MyBatis-Plus 最反直觉的命名——

方法	等价 SQL	匹配
likeLeft(col, "138")	LIKE '%138'	以 138 结尾
likeRight(col, "138")	LIKE '138%'	以 138 开头
like(col, "138")	LIKE '%138%'	包含 138

记忆法：likeLeft 的 "Left" 是指百分号在左边（'%138'），不是"匹配左边"。

实战：

- 查 phone 以 138 开头 → likeRight("phone", "138") ✓
- 查 email 以 @qq.com 结尾 → likeLeft("email", "@qq.com") ✓
- 查 name 包含"小" → like("name", "小") ✓

A.9 ⚠ 经典陷阱：.list() 不接参数

```
.list()           // ☒ 正确：返回所有匹配记录  
.list(5)         // ☒ 错误：方法签名不接参数
```

MyBatis-Plus 的 .list() 没有"限制条数"的重载。要限制条数有两种姿势：

姿势 A: `.last()` 拼 LIMIT

```
.orderByDesc("create_time")
.last("LIMIT 5")           // ← 在末尾追加任意 SQL
.list();
```

姿势 B: `.page()` 分页

```
Page<User> page = new Page<>(1, 5);
userService.query()
    .eq("status", 1)
    .orderByDesc("create_time")
    .page(page);

List<User> users = page.getRecords();
```

简单需求用 `.last("LIMIT 5")`，需要分页用 `.page()`。

A.10 ⚠️ `last()` 的 SQL 注入风险

```
.last("LIMIT 5")           // ✅ 字面量，安全
.last("LIMIT " + userInputLimit) // ❌ 拼用户输入，SQL 注入！
```

`last()` 不做参数校验，直接拼到 SQL 末尾。绝对不能拼用户输入。

生产规则: `last()` 只用来拼字面量片段（LIMIT 5、FOR UPDATE、FOR SHARE、OFFSET 100 等）。

A.11 面试 Q&A

Q1: `save` 和 `saveOrUpdate` 区别？

方法	主键有值	主键无值
<code>save</code>	INSERT（可能主键冲突）	INSERT
<code>saveOrUpdate</code>	UPDATE	INSERT

Q2: `one()` 查到多条会怎样？

抛 `TooManyResultsException`。如果允许多条，用 `.list().get(0)` 或加 `.last("LIMIT 1")`。

Q3: MyBatis-Plus 的分页是怎么实现的？

通过 `PaginationInnerInterceptor` 拦截 SQL，自动加 **LIMIT/OFFSET**。底层是 MyBatis 的插件机制（基于 JDK 动态代理）。

Q4：链式调用是怎么实现的？

每个条件方法返回 `this`（wrapper 对象自身），所以能 `.eq().gt().like()...` 一直串下去。这是 **Builder 设计模式 + 流式接口（Fluent Interface）** 的经典实现。

✦ **加分：**和 Spring 的 `RestTemplate.exchange().retrieve().bodyToMono()`、Java 8 Stream `.filter().map().collect()` 是同一种模式。

Q5：如果业务规则是“stock 是 decimal 类型”，`gt(stock, 0)` 和 `ge(stock, 1)` 还等价吗？

不等价：

- `stock = 0.5` 时，`gt(0)` 通过，`ge(1)` 不通过
- 当前项目 stock 是 int，所以二者等价；如果改成 decimal 就有差异

能答出“数据类型决定等价性”是面试加分点。

A.12 自检练习题答案

题目：查询 `tb_user` 表里所有 `status = 1` 且 `phone` 以 `138` 开头的用户，按 `create_time` 倒序排，最多取 5 条。

字符串版：

```
userService.query()
    .eq("status", 1)
    .likeRight("phone", "138")           // ★ likeRight = '138%'
    .orderByDesc("create_time")
    .last("LIMIT 5")                     // ★ 限制 5 条
    .list();
```

Lambda 版（推荐）：

```
userService.lambdaQuery()
    .eq(User::getStatus, 1)
    .likeRight(User::getPhone, "138")
    .orderByDesc(User::getCreateTime)
    .last("LIMIT 5")
    .list();
```

翻译成 SQL：

```
SELECT * FROM tb_user
WHERE status = 1 AND phone LIKE '138%'
```

```
ORDER BY create_time DESC
LIMIT 5;
```

A.13 一张图总结

```
起手
service.update() / query() / lambdaUpdate() / lambdaQuery()
    ↓
配置
.eq() / .gt() / .ge() / .like() / .in() / .isNull() ...
.setSql() / .set()          (仅 update 用)
.orderByAsc() / .orderByDesc() / .last() ...
    ↓
终止
查询: .one() / .list() / .count() / .page() / .exists()
更新: .update() / .remove()
```

✅ 自我检验清单（附录 A）

学完 MyBatis-Plus 这部分，你应该能：

- ☐ 说出“链式调用 4 步公式”（起手 / 配置 / 终止）
- ☐ 默写 `eq` / `ne` / `gt` / `ge` / `lt` / `le` 6 个比较方法
- ☐ 区分 `like` / `likeLeft` / `likeRight` 的方向
- ☐ 区分 `.set()` 和 `.setSql()` 的使用场景
- ☐ 解释 `.list()` 和 `.last("LIMIT 5") + .list()` 的差别
- ☐ 说出 `last()` 的 SQL 注入风险
- ☐ 写出 Lambda 版本的查询并说明类型安全的好处
- ☐ 翻译 A.12 自检练习题（不看答案）
- ☐ 解释链式调用的实现原理（每个方法返回 `this`）

附录 A 完。下一站：章节 6.2 Redisson + 一人一单。



附录 B：秒杀核心（VoucherOrderServiceImpl）

整理自章节 6 系列对话。本附录覆盖 6.1 + 6.2 全部内容。6.3（AopContext + 事务代理）会在另外的笔记中整理。

B.1 秒杀业务场景

业务定义

店家发布活动：

"10 元代金券抢购，原价 50 元，限量 100 张，10:00 开抢！"

用户行为：

9:59:59 → 用户疯狂刷页面

10:00:00 → 100 万人同时点击"立即抢购"

10:00:01 → 100 张券被抢光

10:00:02 → 用户晒晒晒

技术挑战

挑战	量级
瞬时高并发	100 万 QPS
数据一致性	不能超卖（卖出 101 张就亏了）
公平性	一人一单（防黄牛）
快速响应	用户等 5 秒就跑了

涉及的两张表

```
-- 秒杀活动信息
tb_seckill_voucher (voucher_id PK, stock, begin_time, end_time)

-- 订单
tb_voucher_order (id, user_id, voucher_id, status, create_time)
```

B.2 演进史（注释代码的化石价值）

`VoucherOrderServiceImpl.java` 里保留了 3 版迭代：

版本	位置	核心做法	问题
V1 注释 88-118 行	无锁直接扣库存	<code>UPDATE stock = stock - 1</code>	超卖
V2 注释 151-190 行	<code>SimpleRedisLock</code>	自研 Redis 锁	误删、不可重入、不能续期
V3 活跃版本 50-85 行	<code>Redisson</code> 锁	可重入 + 看门狗	当前最优

🔴 面试金句：

"这个项目的秒杀实现我做过 3 版迭代——V1 无锁直接扣库存（发现超卖）→ V2 自研 `SimpleRedisLock`（发现误删问题）→ V3 引入 `Redisson`（解决可重入和续期）。每一步都是踩坑后的演进。"

B.3 V1 的超卖 Bug

代码（已注释）

```
SeckillVoucher voucher = seckillVoucherService.getById(voucherId);
if (voucher.getStock() < 1) return Result.fail("库存不足");
boolean success = seckillVoucherService.update()
    .setSql("stock = stock - 1")
    .eq("voucher_id", voucherId)
    .update();
```

高并发时序图（库存=1，两用户同抢）

时刻	Thread A（用户1001）	Thread B（用户1002）	DB stock
T1	查库存 → 1		
T2		查库存 → 1	
T3	判断 1 >= 1 → 通过		
T4		判断 1 >= 1 → 通过	
T5	UPDATE stock = stock - 1		0
T6		UPDATE stock = stock - 1	-1 ❌
T7	插入订单（A 抢到）		
T8		插入订单（B 也抢到）	

结果：库存只有 1 张，但卖出 2 张 → 超卖

根本原因：查库存 → 判断 → 扣减 三步不是原子操作，中间被其他线程插入。

B.4 超卖的 4 种解决方案

方案	正确性	性能	集群支持	复杂度	适用场景
synchronized	✅ 单机	❌ 串行	❌	★	单机低并发
DB 悲观锁（SELECT...FOR UPDATE）	✅	❌ 锁等待长	✅	★★	中并发
DB 乐观锁（CAS） ★	✅	✅	✅	★★★	教学项目推荐
Redis 预扣 + 异步落库	✅	✅✅	✅	★★★★★	真实秒杀

B.5 黑马点评的乐观锁实现

V3 实际使用的扣减逻辑

```
boolean success = seckillVoucherService.update()
    .setSql("stock = stock - 1")
    .eq("voucher_id", voucherId)
    .gt("stock", 0) // ★ 关键：增加 "stock > 0" 条件
    .update();
```

对应 SQL:

```
UPDATE tb_seckill_voucher
SET stock = stock - 1
WHERE voucher_id = ? AND stock > 0; -- ★ WHERE 加条件
```

高并发下的执行（不再超卖）

时刻	Thread A	Thread B	DB stock
T1	UPDATE...WHERE stock>0		
	受影响行数 = 1 ✓ 抢到		1 → 0
T2		UPDATE...WHERE stock>0 因为 stock=0, 不满足 WHERE 受影响行数 = 0 ✗ 没抢到	

关键魔法：UPDATE ... WHERE 由 MySQL 在 InnoDB 引擎下原子执行——WHERE 条件检查和 SET 修改两步在同一行锁内完成，绝不会中途被打断。

✦ 这就是经典的 **CAS（Compare And Swap）思想**——用条件判断代替互斥锁。

简化版乐观锁的"含蓄设计"

经典乐观锁基于版本号：

```
UPDATE table SET stock = stock - 1, version = version + 1
WHERE id = ? AND version = ?
```

黑马点评**简化**——不用 version 号，直接用 `stock > 0` 当条件。

为什么能简化？ 因为 stock 本身就是单调递减的状态，"stock > 0" 暗含了"还有库存可扣"的状态校验。

B.6 抽查重点

Q1：库存 stock=5，10 个用户同时点 V1 无锁版，最多卖几张？

A: 最多 10 张（极端情况下所有 10 个线程在 update 之前都查到 stock=5，都通过判断，都各自 -1，最终 stock=-5）。

Q2: `gt("stock", 0)` 和 `ge("stock", 1)` 等价吗？

A: 对 **int 类型 stock 完全等价**；对 **decimal/float 类型不等价**（0.5 会让 gt 通过、ge 不通过）。当前项目 stock 是 int，所以二者等价。能答出"数据类型决定等价性"是加分点。

Q3: 乐观锁能防"一人一单"吗？

A: **不能**。乐观锁的 WHERE 条件只看 stock，不知道是谁来扣的。同一用户 100 个并发请求都能通过 stock > 0 检查，结果该用户抢到 100 张券。需要分布式锁（章节 6.2）配合。

 章节 6.2：Redisson 分布式锁 + 一人一单

B.7 乐观锁防不住一人一单（回应 Q3）

根因：两种约束的本质不同

维度	防超卖	防一人一单
约束的对象	数据本身（stock 字段）	用户行为（user 维度）
检查的内容	"DB 还允许扣吗"	"这个用户已经下过单吗"
用什么实现	DB 行锁 / 乐观锁	分布式锁 / 唯一索引

关键洞察：

乐观锁的 WHERE 条件只看 stock，不知道也不关心是谁来扣的。它能保证"卖出张数 ≤ 库存"，但保证不了"任意用户的订单数 ≤ 1"。

一人一单的 3 种方案

方案	做法	评价
A: DB 唯一索引	<code>ALTER TABLE tb_voucher_order ADD UNIQUE KEY uk_user_voucher (user_id, voucher_id)</code>	✅ 100% 可靠（DB 兜底）、不用改业务代码
B: 先查后插	<code>query().eq(...).count() > 0 ? fail : insert</code>	❌ 并发下"查+插"非原子，仍会重复
C: 分布式锁	<code>redissonClient.getLock(...)</code> 包裹查+判+插	✅ 黑马点评选这个

生产推荐：A + C 双保险

分布式锁（前置拦截，性能好）

↓

DB 唯一索引（最后兜底，绝对可靠）

面试金句：

"我用分布式锁做前置串行化，但真正的兜底是数据库唯一索引——锁失效或者绕过，DB 也拦得住。两层防御。"

B.8 锁粒度的设计哲学

粒度	key 设计	评价
全局锁	"lock:seckill:global"	❌ 所有人串行，性能崩
按用户锁	"lock:order:" + userId	🟡 同用户不同券也得排队（不必要）
按用户+券 ★	"lock:order:" + userId + ":" + voucherId	✅ 黑马点评选的

设计原则

锁粒度 = 业务唯一性约束。 业务约束是"同一用户对同一券 1 单"，那锁的范围就精确到这两个维度。

业务约束	锁 key 设计
同一用户对同一券 1 单	lock:order:{userId}:{voucherId}
同一用户同时只能进行 1 笔交易	lock:trade:{userId}
同一房间同时只能 1 人编辑	lock:edit:{roomId}

B.9 Redisson 解决了 SimpleRedisLock 的哪些痛点

问题	SimpleRedisLock	Redisson 怎么解决
误删别人的锁	加 UUID + Lua 脚本	默认就有（Hash 里存"UUID + threadId"）
不可重入	同一线程拿不了第二次	✅ 用 Hash 计数实现可重入
业务超时锁失效	没续期机制	✅ 看门狗自动续期
抢锁失败立即失败	不重试	✅ 支持等待时间 + 失败重试
锁竞争性能差	自旋等待	✅ PubSub 唤醒，避免空轮询

引入 Redisson 两步

1. pom.xml 加依赖：

```
<dependency>
  <groupId>org.redisson</groupId>
  <artifactId>redisson</artifactId>
  <version>3.17.7</version>
</dependency>
```

2. 配置 Bean:

```
@Configuration
public class RedissonConfig {
    @Bean
    public RedissonClient redissonClient(){
        Config config = new Config();
        config.useSingleServer()
            .setAddress("redis://127.0.0.1:6379")
            .setPassword("123456");
        return Redisson.create(config);
    }
}
```

✦ 生产配置选项:

- `useSingleServer()` → 单机
- `useClusterServers()` → Redis Cluster
- `useSentinelServers()` → 哨兵模式（生产推荐）
- `useMasterSlaveServers()` → 主从

B.10 Redisson 锁的内部数据结构

你以为的 Redis 锁（SimpleRedisLock 风格）

```
SET lock:order:1001:5 "uuid:threadId" NX EX 30
```

单个 String key，靠 SET NX 实现互斥。

Redisson 的 Redis 锁

Hash 结构:

```
key = "lock:order:1001:5"

field 0:    "abc123:Thread-1"    ← UUID + 线程 ID
value 0:    1                    ← 重入计数
```

```
(重入第二次后)
field 0:    "abc123:Thread-1"
value 0:    2                                ← 计数 +1
```

为什么用 Hash？ 为了支持可重入。

什么是可重入？

```
public void methodA() {
    lock.lock();           // 第一次拿锁
    try {
        methodB();         // 调用别的方法
    } finally {
        lock.unlock();
    }
}

public void methodB() {
    lock.lock();           // 第二次拿同一把锁（同一线程）
    try {
        // 业务
    } finally {
        lock.unlock();
    }
}
```

- `synchronized` 是可重入的——同一线程拿两次同一把锁不会死锁
- SimpleRedisLock **不可重入**——第二次 `setIfAbsent` 失败，自己锁死自己
- Redisson **可重入**——第二次加锁时检查"field 是不是我"，是就计数 +1

Redisson 加锁 Lua 脚本（简化版）

```
-- KEYS[1] = lockKey
-- ARGV[1] = TTL（毫秒）
-- ARGV[2] = "UUID:threadId"

if (redis.call('exists', KEYS[1]) == 0) then
    -- 锁不存在，创建并计数为 1
    redis.call('hincrby', KEYS[1], ARGV[2], 1)
    redis.call('pexpire', KEYS[1], ARGV[1])
    return nil
end

if (redis.call('hexists', KEYS[1], ARGV[2]) == 1) then
    -- 锁存在且是自己的，重入 +1
    redis.call('hincrby', KEYS[1], ARGV[2], 1)
    redis.call('pexpire', KEYS[1], ARGV[1])
    return nil
end
```

```
-- 锁是别人的，返回剩余 TTL
return redis.call('pttl', KEYS[1])
```

情况	行为
锁不存在	新建 Hash，计数 1，返回 nil（成功）
锁存在且 field 是自己	计数 +1，刷新 TTL，返回 nil（成功）
锁存在且 field 是别人	返回剩余 TTL（毫秒），调用方决定等多久

Redisson 解锁 Lua 脚本

```
if (redis.call('hexists', KEYS[1], ARGV[1]) == 0) then
    return nil    -- 不是自己的锁，直接返回
end

local counter = redis.call('hincrby', KEYS[1], ARGV[1], -1)

if (counter > 0) then
    redis.call('pexpire', KEYS[1], ARGV[2])    -- 还有重入次数，刷新 TTL
    return 0
else
    redis.call('del', KEYS[1])    -- 计数到 0，删 key
    redis.call('publish', "channel", "unlock")    -- 发消息唤醒等待者
    return 1
end
```

🌟 **publish 是关键**——这就是 Redisson **PubSub** 唤醒等待者的实现，避免其他客户端空轮询。

B.11 看门狗机制详解

问题铺垫：TTL 该设多久？

方案	缺点
TTL 设很大（10 分钟）	客户端真崩了，锁要等 10 分钟才释放
TTL 设很小（5 秒）	业务执行 6 秒就出事（锁自动释放，并发安全失控）
TTL 适中 + 续期 🌟	复杂，需要后台线程 ← Redisson 选这个

本质矛盾：业务执行时间不可预测——既要"快释放"（防止崩了死锁），又要"够长"（覆盖业务）。

看门狗三句话

1. 加锁时默认 TTL = 30 秒

2. 后台线程每 10 秒（TTL 的 1/3）检查一次：业务还没完？刷新 TTL 回到 30 秒

3. 业务调 `unlock()` 或客户端崩了：看门狗停止 → TTL 不再续期 → 30 秒后自动释放

时间线演示

T0=0s	加锁, TTL=30s	业务开始执行
T1=10s	看门狗检查：业务还在	→ 刷 TTL 回 30s
T2=20s	看门狗检查：业务还在	→ 刷 TTL 回 30s
T3=30s	看门狗检查：业务还在	→ 刷 TTL 回 30s
T4=40s	看门狗检查：业务还在	→ 刷 TTL 回 30s
...		
T?=87s	业务终于跑完, unlock	→ 看门狗停止 + DEL 锁

看门狗的"启用条件" ⚠️ 高频坑

默认不启用！只有 `leaseTime = -1` 时才启用：

```
// ❌ 看门狗不启用 (leaseTime 显式给值)
lock.tryLock(1, 30, TimeUnit.SECONDS);
//           ↑
//           leaseTime = 30s, 到期直接释放, 不续期

// ✅ 看门狗启用 (leaseTime = -1)
lock.tryLock(1, -1, TimeUnit.SECONDS);
//           ↑
//           -1 表示"我不指定持锁时间, 交给看门狗管"

// ✅ 看门狗启用 (lock() 无参版)
lock.lock(); // 内部也是 leaseTime = -1
```

🔴 黑马点评代码用的就是 `-1`：

```
locked = lock.tryLock(1L, -1, TimeUnit.SECONDS);
//           ↑↑
//           启用看门狗
```

B.12 tryLock 三个参数详解

```
boolean tryLock(long waitTime, long leaseTime, TimeUnit unit)
```

参数 1: `waitTime` —— 等多久

取值	含义
0	立即放弃
正数（如 1s）	等指定时间，到期没拿到就放弃
-1	死等（不推荐，可能永久阻塞）

参数 2: `leaseTime` —— 持锁多久

取值	含义
正数（如 30s）	30 秒后强制释放（不管业务完没完）
-1	看门狗自动续期，业务完了再释放

参数 3: `unit` —— 时间单位

`TimeUnit.SECONDS` / `MILLISECONDS` / `MINUTES` 等。

注意：unit 同时作用于 waitTime 和 leaseTime——所以两者必须用同一单位。

lock() 系列 4 个方法对比

方法	waitTime	leaseTime	看门狗	用途
<code>lock()</code>	永久等	-1	✓	必须拿到锁的场景
<code>lock(leaseTime, unit)</code>	永久等	指定值	✗	业务时间已知
<code>tryLock()</code>	立即返回	-1	✓	抢不到就放弃
<code>tryLock(waitTime, unit)</code>	指定	-1	✓	等一会儿再放弃
<code>tryLock(waitTime, leaseTime, unit)</code> ★	指定	指定	leaseTime=-1 时 ✓	最灵活，黑马点评选这个

B.13 看门狗源码原理

续期任务调度（简化版）

```
// io.org.redisson.RedissonLock 内部逻辑

private void scheduleExpirationRenewal(long threadId) {
    ExpirationEntry entry = new ExpirationEntry();

    // 创建一个定时任务，每 internalLockLeaseTime / 3 触发
    // 默认 internalLockLeaseTime = 30s，所以 10s 触发一次
    Timeout task = commandExecutor.getConnectionManager()
        .newTimeout(timeout -> {
            renewExpirationAsync(threadId).onComplete((res, e) -> {
                if (res) {
                    scheduleExpirationRenewal(threadId); // 递归调度下一次
                }
            });
        }, timeout, TimeUnit.MILLISECONDS);
}
```

```
        }
    });
    }, internalLockLeaseTime / 3, TimeUnit.MILLISECONDS);

    entry.setTimeout(task);
}
```

续期 Lua 脚本

```
if (redis.call('hexists', KEYS[1], ARGV[2]) == 1) then
    redis.call('pexpire', KEYS[1], ARGV[1])
    return 1
end
return 0    -- 锁不在了，停止续期
```

隐藏知识点

点	说明
看门狗是 进程级 的	每个 RedissonClient 实例共享一个 HashedWheelTimer
续期间隔 = TTL / 3	默认 30s/3 = 10s，留 2 个续期机会，安全+省的平衡
客户端宕机后多久锁释放	最多 30 秒 （看门狗死了，TTL 自然过期）
看门狗如何停止	<code>unlock()</code> 取消定时任务；或客户端宕机时定时任务跟着进程消失

B.14 实战代码逐行讲解

```
public Result seckillVoucher(Long voucherId) {

    // ① 查询优惠券
    SeckillVoucher voucher = seckillVoucherService.getById(voucherId);

    // ② 时间窗口校验
    if (voucher.getBeginTime().isAfter(LocalDateTime.now())) {
        return Result.fail("秒杀未开始");
    }
    if (voucher.getEndTime().isBefore(LocalDateTime.now())) {
        return Result.fail("秒杀已结束");
    }

    // ③ 获取当前用户 ID（从 ThreadLocal）
    Long userId = UserHolder.getUser().getId();

    // ④ 创建 Redisson 锁对象
    RLock lock = redissonClient.getLock("lock:order:" + userId + ":" + voucherId);

    // ⑤ 尝试获取锁
```

```
boolean locked;
try {
    locked = lock.tryLock(1L, -1, TimeUnit.SECONDS);
} catch (InterruptedException e) {
    return Result.fail("获取锁失败");
}

// ⑥ 抢到锁
if (locked) {
    try {
        // ⑦ 通过代理调用事务方法
        IVoucherOrderService proxy = (IVoucherOrderService)
AopContext.currentProxy();
        return proxy.createVoucherOrder(voucherId);
    } catch (Exception e) {
        throw new RuntimeException(e);
    } finally {
        // ⑧ 释放锁（必须在 finally 里！）
        lock.unlock();
    }
}

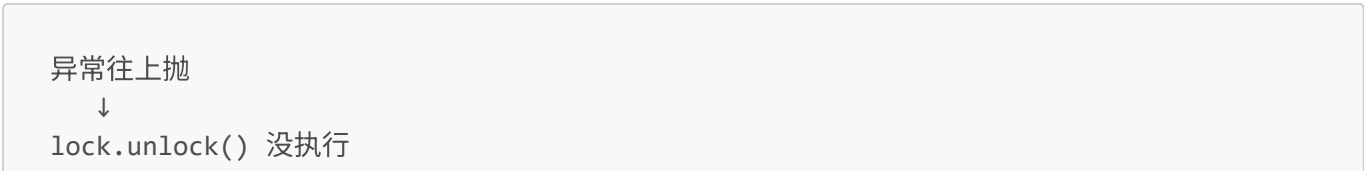
return Result.fail("没抢到");
}
```

8 个设计点

#	点	说明
①	查券	单纯获取活动信息。 优化方向 ：把 voucher 信息预热到 Redis
②	时间窗口	防止刷接口。⚠️ 应用服务器时钟必须同步（NTP）
③	UserHolder.getUser().getId()	来自双拦截器（章节 4）的 ThreadLocal
④	锁 key 设计	粒度 = 用户+券（业务唯一性约束）
⑤	tryLock(1, -1, SECONDS)	等 1s + 启用看门狗
⑥	InterruptedException	建议加 Thread.currentThread().interrupt() 恢复中断状态
⑦	AopContext.currentProxy()	通过代理调用事务方法（ 章节 6.3 主题 ）
⑧	finally { lock.unlock(); }	死锁防御标配 ，否则看门狗一直续期

finally 释放锁的关键性

假设业务抛异常但没 finally：



↓
Redis 里锁还在
↓
看门狗还在续期 ← 30 秒、60 秒、90 秒... 一直续到客户端死
↓
其他用户永远抢不到这把锁 ← 死锁

✦ **finally + unlock** 是分布式锁的标配，没有这个就不要写锁。

B.15 6.2 完整面试 Q&A

Q1: Redisson 的看门狗是怎么实现的？

"**HashedWheelTimer 定时调度 + Lua 续期脚本。**

加锁时如果 leaseTime=-1（默认），Redisson 会启动一个定时任务，每 internalLockLeaseTime/3 秒（默认 10 秒）执行一次 Lua 续期脚本——脚本检查锁是否还属于当前线程，是就 PEXPIRE 回 30 秒。

调 unlock() 时会取消定时任务并删 Redis 锁。客户端宕机时定时任务跟着进程消失，Redis 锁靠 TTL 在最多 30 秒后自动释放——防死锁。"

Q2: tryLock 的三个参数你能解释吗？

"**tryLock(waitTime, leaseTime, unit)。**

waitTime 是等待获取锁的最大时间——抢不到等多久。leaseTime 是持锁时间——-1 表示交给看门狗管，否则到期硬释放。unit 是时间单位，同时作用于前两个参数。

黑马点评用 (1, -1, SECONDS)——等 1 秒抢，启用看门狗。秒杀场景用户最多等 1 秒，业务时间不可预测用看门狗最稳。"

Q3: 为什么必须用 finally 释放锁？

"防死锁。如果业务抛异常但锁没释放，看门狗会一直续期——这把锁就再也没人能拿到，除非客户端宕机。所有分布式锁都必须 try { ... } finally { lock.unlock(); }，这是标配。"

Q4: 捕获 InterruptedException 后该做什么？

"**调 Thread.currentThread().interrupt() 恢复中断状态。**捕获后不恢复，上层代码就不知道线程被中断过——破坏了 Java 中断协作模型。"

Q5: Redisson 锁和 SimpleRedisLock 比，多了什么？

"1. **可重入**——Hash 计数实现，同一线程能反复加锁 2. **看门狗续期**——业务超时锁不会被强释 3. **PubSub 唤醒**——锁释放时主动通知等待者，不空轮询 4. **失败重试机制**——抢不到锁可以等指定时间再试"

Q6: Redisson 的锁是怎么实现的？

"Redisson 用 **Redis Hash 结构** 存锁状态，**field** 是 'UUID + 线程 ID' 的组合，**value** 是重入计数。这个设计支持两个特性：

1. **可重入**：同一线程再次加锁时，Lua 脚本检查 field 匹配，计数 +1 而不是阻塞
2. **跨进程唯一标识**：UUID 标识 JVM，threadId 标识线程内位置，集群下也不会撞

加锁释放都通过 **Lua 脚本原子执行**——多步操作（exists、hincrby、pexpire）一次性完成。

释放时计数减到 0 才真删 key，并 **publish 一条消息** 到 channel，**唤醒所有等待这把锁的客户端**——避免其他客户端做空轮询。

加上还有看门狗自动续期机制，所以是**生产级分布式锁的事实标准**。"

B.16 章节 6 知识闭环图

秒杀业务场景

↓

V1 无锁版 → 超卖 Bug

↓

解决方案 1: 乐观锁 (CAS, stock > 0)

↓ 但只防超卖，不防一人一单

解决方案 2: 分布式锁

↓

V2 SimpleRedisLock → 5 大痛点

↓

V3 Redisson

- ├─ Hash + Lua 实现可重入
- ├─ 看门狗 (TTL/3 续期) 解决业务超时
- ├─ PubSub 唤醒等待者
- └─ try-finally + unlock 防死锁

↓

还有一个伏笔：AopContext.currentProxy() 为什么必要？

↓

章节 6.3: Spring AOP 代理 + @Transactional 失效场景

✅ 章节 6 自我检验清单

学完 6.1 + 6.2，你应该能：

- ☐ 用一句话描述超卖问题的根因（三步非原子）
- ☐ 默写 4 种防超卖方案，能说出各自优劣
- ☐ 解释黑马点评简化版乐观锁为什么不用 version 字段
- ☐ 解释"乐观锁防超卖，分布式锁防一人一单"是两件事
- ☐ 说出"A 唯一索引 + C 分布式锁"双保险方案
- ☐ 描述 Redisson 锁的 Hash 数据结构（field + value 含义）
- ☐ 解释为什么 Redisson 是可重入的（Hash 计数）
- ☐ 看门狗启用条件（leaseTime = -1）
- ☐ tryLock 三参数语义

- ☐ 描述客户端宕机后锁多久释放（最多 30 秒）
 - ☐ 解释为什么必须 finally + unlock（死锁防御）
 - ☐ InterruptedException 处理的"恢复中断状态"细节
-

附录 B 完。下一站：章节 6.3 AopContext.currentProxy() + @Transactional 失效场景。